

Programmazione 1

Corso c

29/10/2023

> La ricorsione su array (2)

Alessandro Mazzei

alessandro.mazzei@unito.it

Dipartimento di Informatica

Università degli Studi di Torino



UNIVERSITÀ
DI TORINO

Outline

- Array e ricorsione
- Print e Somma di un array
- Ricorsione ControVariante, CoVariante, Dicotomica

Outline

- Array e ricorsione
- Print e Somma di un array
- Ricorsione ControVariante, CoVariante, Dicotomica

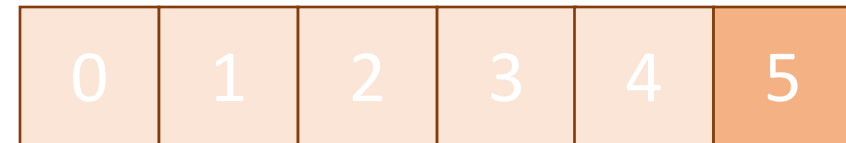
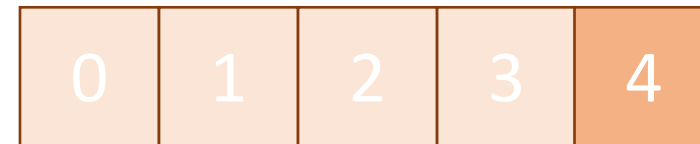
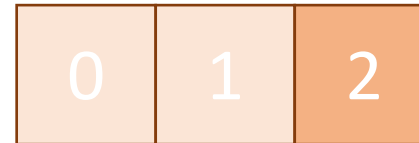
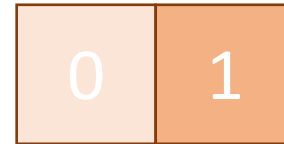
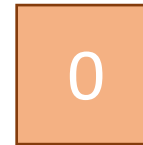
Array come strutture ricorsive

- La ricorsione è utilissima se:
 - Il problema è ricorsivo (fattoriale, Hanoi)
 - La struttura dati che uso per risolverlo è ricorsiva
- Ma un array è ricorsivo?

Array come struttura ricorsiva

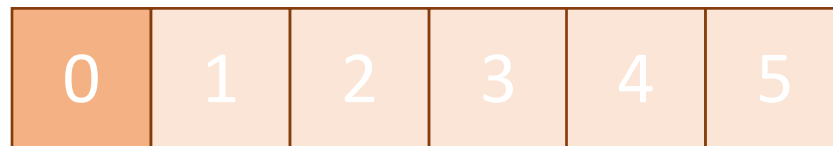
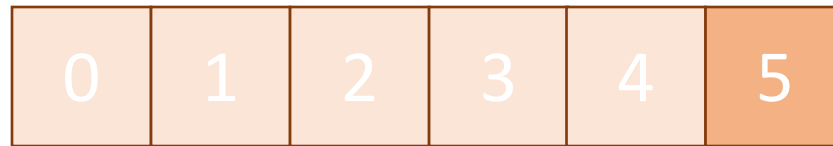
Possibile definizione ricorsiva di array:

- Un array è una cella indicizzata
- Un array è una cella indicizzata seguita da un array



Array come struttura ricorsiva

Nelle chiamate ricorsive useremo degli indici variabili per richiamare la funzione su un problema più semplice, ovvero su un array più piccolo



Outline

- Array e ricorsione
- Print e Somma di un array
- Ricorsione ControVariante, CoVariante, Dicotomica

Stampa array ricorsiva

```
void printArray(int arLen, int ar[]) {  
    //wrapper, guscio  
    printArrayRic(arLen, ar, 0);  
}
```

```
void printArrayRic(int arLen, int ar[], int index) {  
    if(index == arLen) {}  
    else  
    {  
        printf("%d ", ar[index]);  
        printArrayRic(arLen, ar, index+1);  
    }  
}
```


Stampa array ricorsiva

```
void printArray(int arLen, int ar[]) {  
    //wrapper, guscio  
    printArrayRic(arLen, ar, 0);  
}
```

```
void printArrayRic(int arLen, int ar[], int index) {  
    if(index != arLen) {  
        printf("%d ", ar[index]);  
        printArrayRic(arLen, ar, index+1);  
    }  
}
```

Somma array ricorsiva

```
int sumArray(int arLen, int ar[]) {  
    return sumArrayRic(arLen, ar, 0);  
}  
  
void sumArrayRic(int arLen, int ar[], int index) {  
    ret;  
    if(index == arLen) {  
        ret = 0;  
    }  
    else {  
        ret = ar[index] + sumArrayRic(arLen, ar, index+1);  
    }  
    return ret;  
}
```

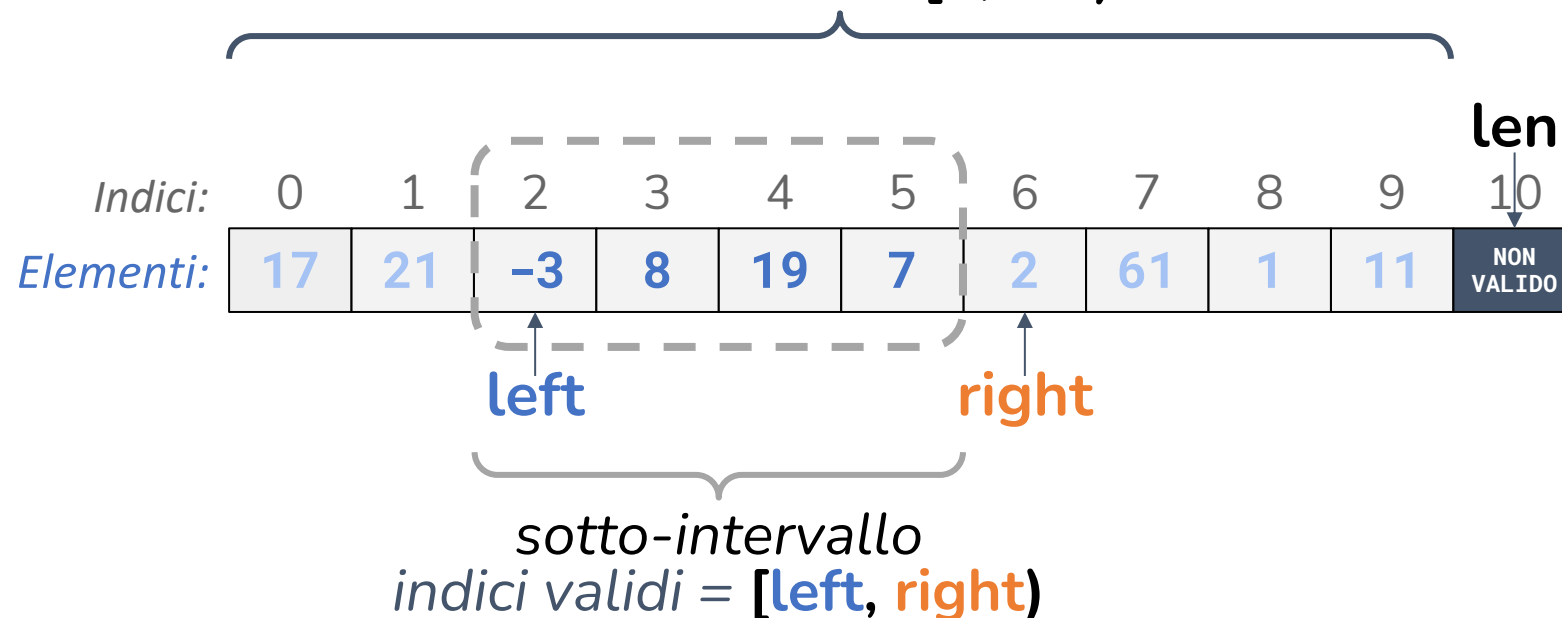
Outline

- Array e ricorsione
- Print e Somma di un array
- Ricorsione ControVariante, CoVariante, Dicotomica

Intervalli di array e ricorsione

len elementi inizializzati
indici validi = $[0, \text{len})$

- L'intervallo aperto ci permette di identificare un intervallo (*sottoarray*) compreso tra le posizioni *left* (incluso) e *right* (escluso).
- Avremmo 3 diverse forme di ricorsione su array a seconda di come variano *left* e *right*

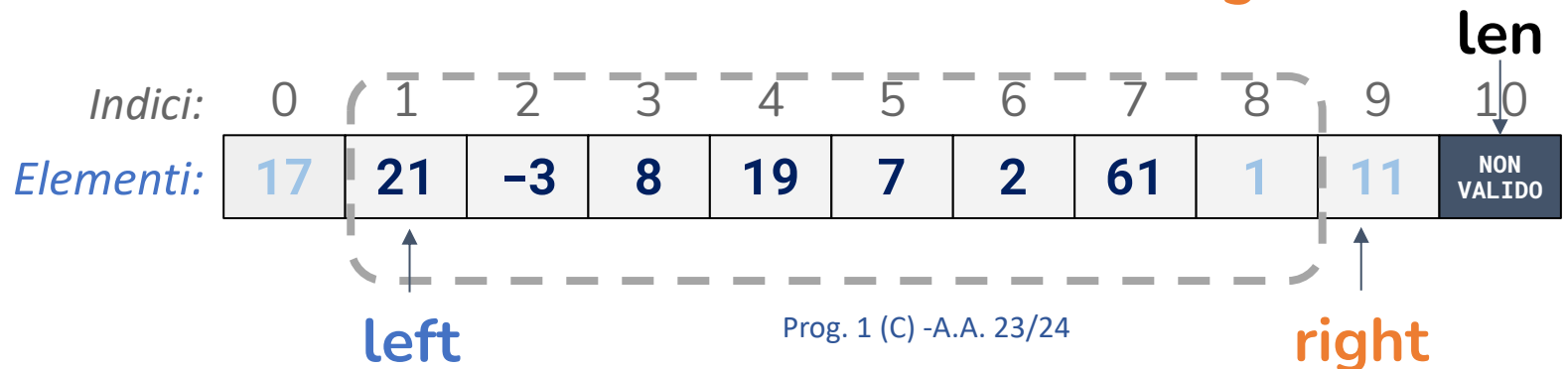
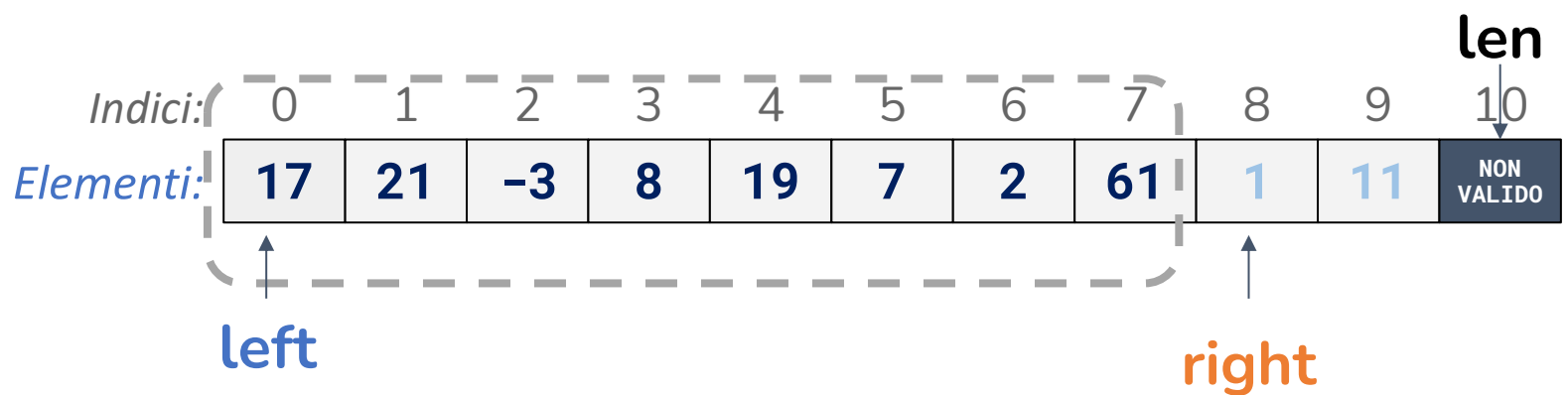
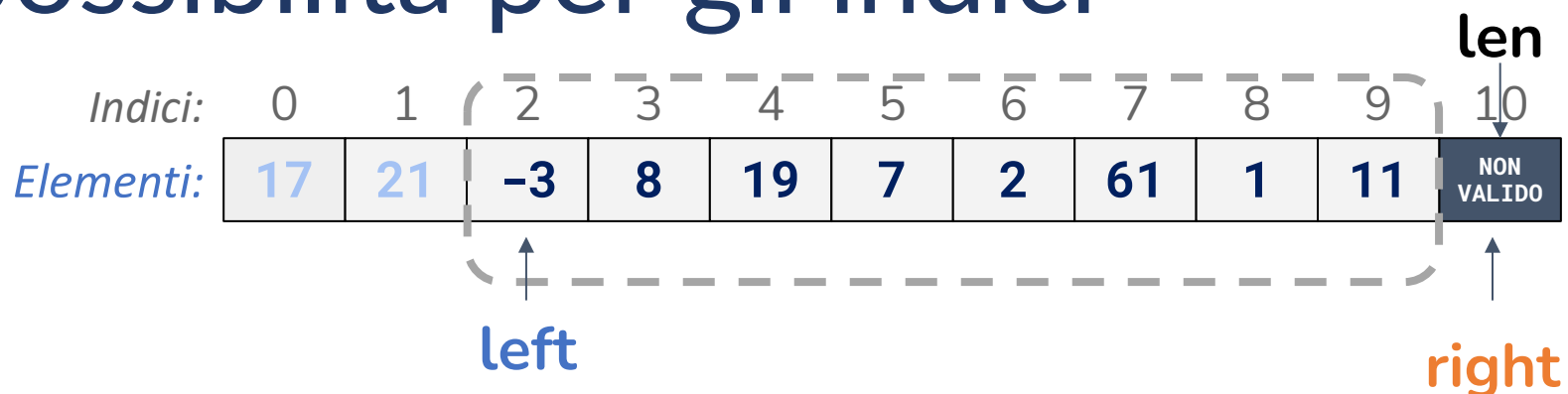


Caso base: $[\text{left}, \text{right}) == [0, \text{len})$

Caso generale: $[\text{left}, \text{right}) \subseteq [0, \text{len})$

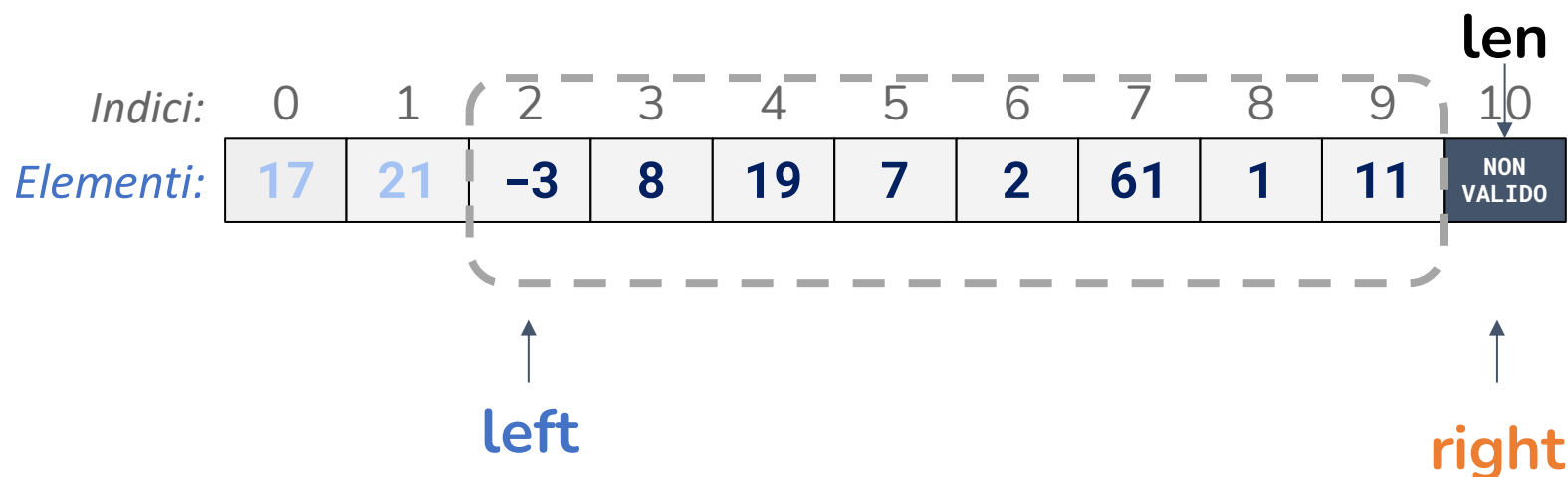
cioè vale: $0 \leq \text{left} \leq \text{right} \leq \text{len}$

3 possibilità per gli indici



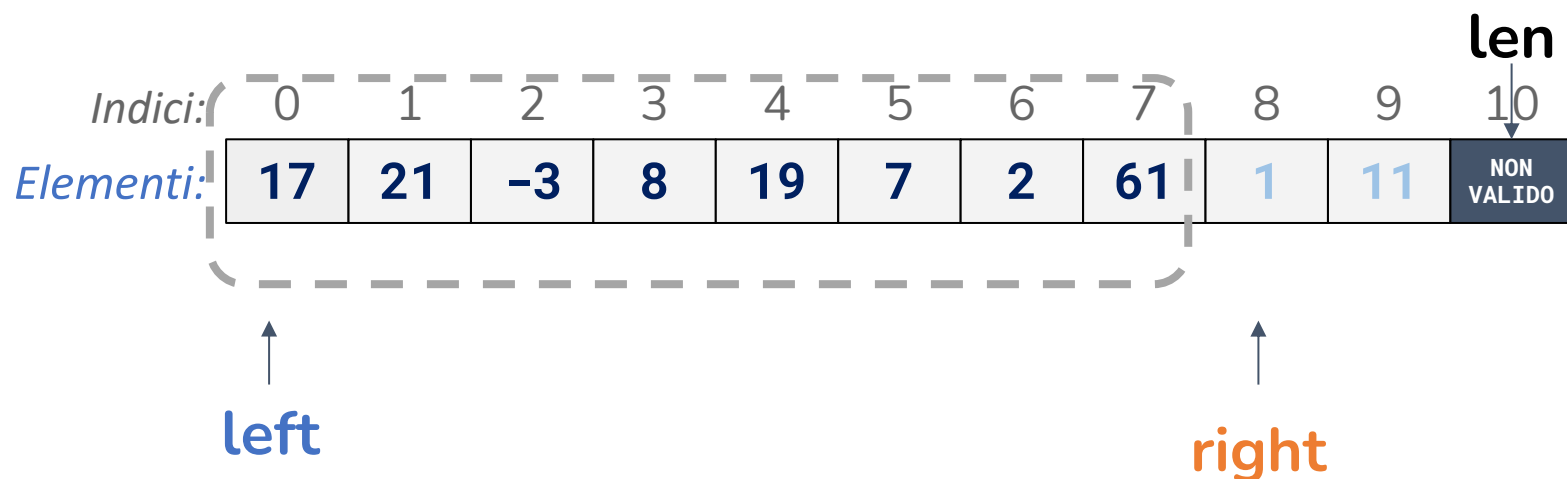
Ricorsione controvariante (leftToRight)

- right rimane costante a len
- Quando $\text{left} == \text{right} == \text{len}$ sono nel caso base, ovvero intervallo vuoto
- left parte da 0 e viene incrementato ad ogni chiamata ricorsiva, diminuendo la lunghezza dell'intervallo considerato



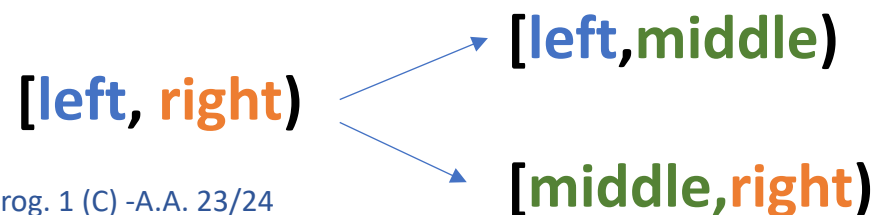
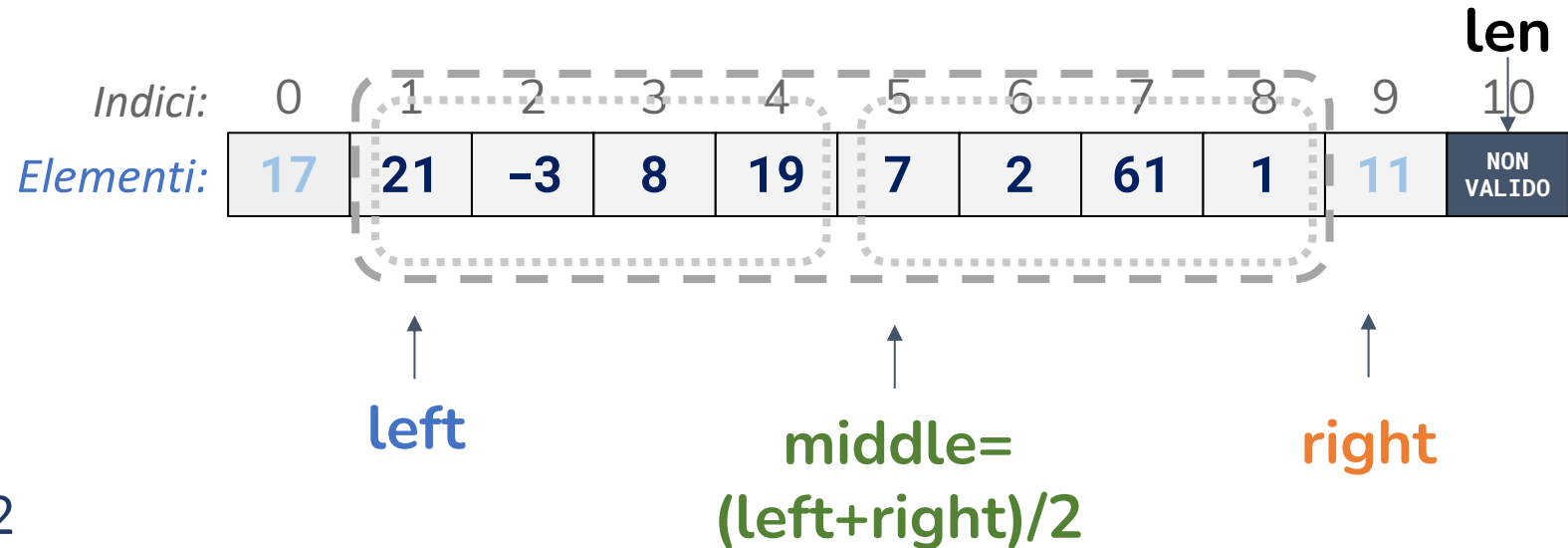
Ricorsione covariante (rightToLeft)

- left rimane costante a 0
- Quando $\text{right} == \text{left} == 0$ sono nel caso base, ovvero sottoarray vuoto
- right parte da len e viene decrementato ad ogni chiamata ricorsiva, diminuendo la lunghezza dell'intervallo considerato



Ricorsione dicotomica (leftToRight & rightToLeft)

- right parte da len e viene decrementato
- left parte da 0 e viene incrementato
- Costruisco due nuovi intervalli usando il punto medio e faccio 2 chiamate ricorsive
- Quando $\text{right} == \text{left} + 1$ sono nel caso base, ovvero sottoarray di grandezza 1



Esercizio

Calcolare il numero di interi pari presenti in un array con ricorsione controvariante, covariante e dicotomico.

Interi pari nell'array: controvariante

Assumiamo che la chiamata del metodo ricorsivo

`contaPariCon(len, a, left)` restituisca il numero di interi pari nel segmento `a[left...len)`. Osserviamo che gli interi pari nel segmento `a[left...len)` sono:

- gli stessi di quelli dell'intervallo `a[left+1...len)` se `a[left]` è dispari;
- uno in più di quelli dell'intervallo `a[left+1...len)` se `a[left]` è pari.

Interi pari nell'array: covariante

Assumiamo che la chiamata del metodo ricorsivo

`contaPariCov(len, a, right)` restituisca il numero di interi pari nel segmento $a[0 \dots \text{right})$. Osserviamo che gli interi pari nel segmento $a[0 \dots \text{right})$ sono:

- gli stessi di quelli dell'intervallo $a[0 \dots \text{right}-2)$ se $a[\text{right}-1]$ è dispari;
- uno in più di quelli dell'intervallo $a[0 \dots \text{right}-2)$ se $a[\text{right}-1]$ è pari.

Interi pari nell'array: dicotomico

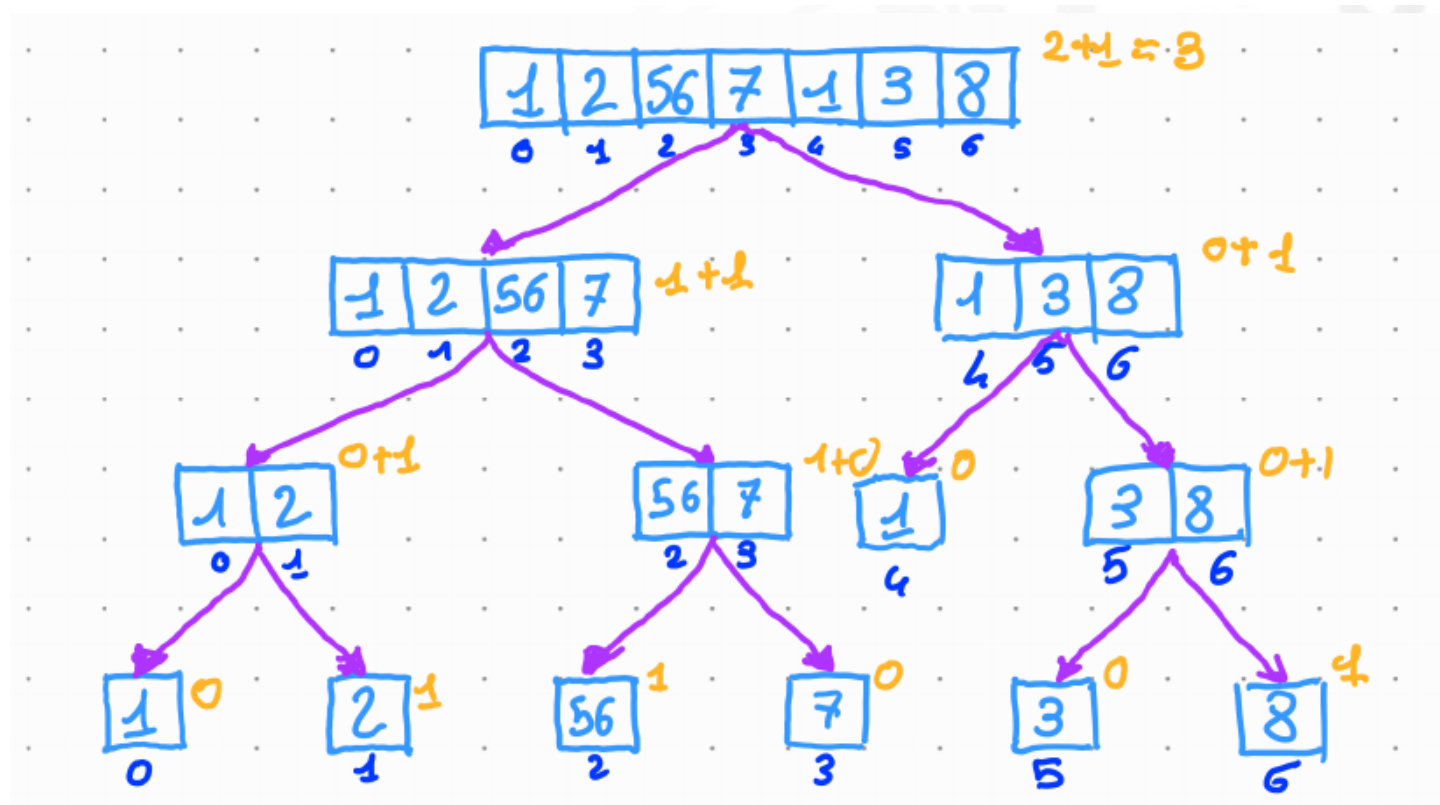
Assumiamo che le due chiamate ricorsive del metodo,

`contaPariDic(len, a, l, (l+r)/2)` e

`contaPariDic(len, a, (l+r)/2, r)`, restituiscano i numeri degli interi pari presenti nei rispettivi segmenti. Osserviamo che gli interi pari nel segmento $a[l...r)$ sono la somma di due quantità:

- gli interi pari presenti nell'intervallo $a[l...(l+r)/2)$;
- gli interi pari presenti nell'intervallo $a[(l+r)/2, r)$.

Interi pari nell'array: dicotomico



Esercizio

- Trovare il valore minimo in un array usando la ricorsione
Controvariante, Covariante e Dicotomica